

Agent Task Control Plane

English | [简体中文](#)

A Reference Architecture for Human-Governed Multi-Agent Task Control

Version: v0.1

Status: Draft / Open for Discussion

Language: English

Author: Community Proposal

License Recommendation: CC BY 4.0 for this whitepaper; Apache-2.0 or MIT for a future reference implementation

Abstract

As AI agents evolve from single-task, single-session tools into systems that execute multiple long-running tasks concurrently and asynchronously, a new bottleneck emerges: **agent parallelism can scale quickly, but human working memory, context-switching capacity, and attention do not scale with it.**

In practical multi-agent coding workflows, a person soon encounters recurring problems:

- forgetting the current stage of a task;
- not knowing whether code has been deployed;
- not knowing whether a deployment has been verified;
- losing track of which task changed a page or file;
- contaminating one task with another task's context;
- allowing an agent to declare success after completing only its local step;
- manually acting as router, project manager, state machine, observer, message bus, and approval gate;
- losing the benefit of agent parallelism because cognitive overhead grows with the number of tasks.

This whitepaper proposes a reference architecture:

Agent Task Control Plane (ATCP): a task control plane for multi-agent execution.

Its purpose is not to create more agents. Its purpose is to move the following responsibilities out of human working memory and into the system:

- task routing;
- task-state management;
- agent coordination;
- execution observation;
- human approval;
- outcome verification;
- end-to-end traceability;
- human-attention routing.

The central claim is:

Scaling agents should not scale human cognitive load.

1. Background

1.1 From One Agent to Parallel Agents

With a single agent, the user normally maintains one active context:

```
graph TD
    Human --> Agent
    Agent --> Result
```

When agents support long-running, asynchronous, and parallel work, the operating model becomes:



This appears to increase parallel capacity. In practice, however, the human must continuously remember:

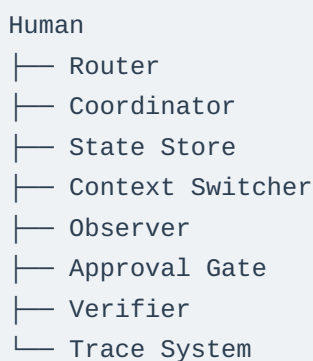
- the goal of Agent A;
- the current state of Agent B;
- the failure reason for Agent C;
- which agent is still running;
- which agent has completed its local work;
- which task still needs deployment;
- which deployment still needs verification;
- which tasks depend on or conflict with one another;
- which instruction belongs to which context.

The first real bottleneck of a multi-agent system may therefore be:

Human Context Management

1.2 Humans Are Performing System Responsibilities

Without a unified task-control layer, the human effectively becomes:



This structure does not scale.

As parallel tasks increase from one to three, five, or ten, the operator repeatedly performs the following cycle:

```
Unload Task A context
↓
Reload Task B context
↓
Reconstruct Task B state
↓
Make one decision
↓
Switch to Task C
↓
Recover Task C history and constraints
```

Context switching itself becomes a major cost.

2. Problem Definition

ATCP is not primarily concerned with how to launch more agents. It asks:

How can one person govern multiple asynchronous agents while keeping only a small number of current decisions in mind?

2.1 Context Contamination

A user may accidentally send a message intended for Task A to the agent working on Task B.

Instead of recognizing that the instruction is likely misplaced, an agent may reinterpret it within its current context. The result is silent context contamination.

2.2 State Loss

Common questions include:

```
Was the code changed?  
Did the build pass?  
Was it deployed?  
Was deployment verified?  
Who changed it?  
Which task caused the change?
```

If these questions can be answered only by searching chat history or relying on human memory, the multi-agent system cannot truly scale.

2.3 Local Completion Is Not Task Completion

A coding agent may reason:

```
Code changed  
→ Task complete
```

But the real task may require:

```
Code complete  
+ Review  
+ Build  
+ Deploy  
+ Verify  
= DONE
```

Therefore, **DONE** must not be merely a natural-language declaration made by an agent.

2.4 Goal Drift

An original goal might be:

```
Deploy an already completed frontend change
```

During execution, the work may gradually turn into:

```
Investigate SSH
→ Change configuration
→ Replace the login method
→ Continue debugging the environment
```

The agent's current actions can drift away from the original goal even when each individual action appears locally reasonable.

2.5 Excessive Human-Attention Cost

If twenty agents are running and the user must continuously monitor twenty tasks, the system has achieved computational parallelism without achieving organizational leverage.

An effective system should instead reduce the user's current working set:

```
20 tasks
↓
15 operating normally
3 recovering automatically
2 requiring a human decision
↓
Show only those 2 to the user
```

3. Design Goals

3.1 Task-First, Not Chat-First

The system's first-class entities should be:

```
Task
State
Event
Artifact
Approval
Trace
```

Chat is only one interface through which task input can arrive.

3.2 Authoritative State

Task state must not depend solely on an agent's description. The system requires an explicit:

Source of Truth

3.3 Deterministic Logic Should Use Software Rules

Problems that can be governed by a state machine should not be delegated entirely to probabilistic model judgment.

For example:

```
DEPLOYED
→ must enter VERIFYING
→ cannot become DONE before verification
```

This is a deterministic rule.

Language-model reasoning is more appropriate for questions such as:

```
Does this user message belong to Task A or Task B?
```

or:

```
Has the agent significantly drifted from the original goal?
```

3.4 Context Isolation by Default

Agents should not automatically share complete conversation histories.

Inter-agent communication should prefer:

- Task IDs;
- structured state;
- events;
- artifacts;
- explicit constraints;
- minimum necessary context.

3.5 Human-in-the-Loop as a State-Machine Concept

Human review should not be an external pop-up disconnected from task state. It should be modeled as a formal state, for example:

WAITING_HUMAN_APPROVAL

3.6 Human Attention Is Scarce

Normal operation should not continuously interrupt the user. Human attention should be reserved for:

- high-risk actions;
 - conflicting directions;
 - failures that cannot recover automatically;
 - important approvals;
 - visual or business judgment;
 - inconsistent agent states;
 - prolonged stalls;
 - suspected goal drift.
-

4. Core Abstractions

ATCP defines six foundational abstractions.

4.1 Task

A Task is the system's primary unit of work.

At minimum, it contains:

```
task_id: T-1024
goal: "Fix the player description on the recording page and release it"
owner: human
priority: normal
created_at: ...
desired_state: VERIFIED
observed_state: EXECUTING
```

4.2 State

State identifies the task's current position in its lifecycle. State is governed by the State Machine Manager.

4.3 Event

Every significant action produces an Event, such as:

```
TASK_CREATED
AGENT_ASSIGNED
CODE_CHANGED
BUILD_PASSED
DEPLOY_STARTED
DEPLOY_FINISHED
VERIFY_FAILED
HUMAN_APPROVED
TASK_REOPENED
```

Events form the task's history.

4.4 Artifact

An Artifact is a verifiable output produced during execution, including:

- code diffs;
- commits;
- build artifacts;
- deployment versions;
- test reports;
- screenshots;
- logs;
- data-analysis results;
- documents.

4.5 Approval

An Approval records authorization that must be granted by a human or another designated role.

4.6 Trace

A Trace connects an intent to its final verified result:

Human Intent



Task ID



Agent



Diff



Commit



Build



Deploy



Verify

The goal is simple:

From any result, the system should be able to explain why it exists.

5. Core Roles

5.1 Router

The Router decides which Task should receive an input, or whether a new Task should be created.

It answers: **Where should this go?**

5.2 Coordinator

The Coordinator manages dependencies, concurrency, mutual exclusion, conflicts, and priorities across Tasks and agents.

It answers: **Who acts first, who waits, and what can run in parallel?**

A sequential flow might be:

Coding



Review



Build

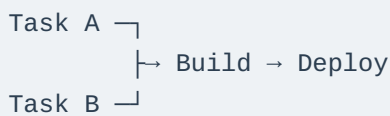


Deploy



Verify

A converging flow might be:



5.3 State Machine Manager

The State Machine Manager owns valid states, transition conditions, and the Definition of Done.

It is the authoritative state source. An execution agent should not have unilateral permission to mark a task **DONE**.

5.4 Observer

The Observer continuously compares agent behavior with real system state. It detects:

- repeated failures;
- unproductive loops;
- goal drift;
- prolonged stalls;
- state conflicts between agents;
- discrepancies between declared and observed state;
- abnormal context;
- unverified assumptions.

The Observer is not the primary executor.

5.5 Executor Agent

Executor Agents perform specific work, for example:

- coding;
- testing;
- deployment;
- research;
- data analysis.

5.6 Verifier

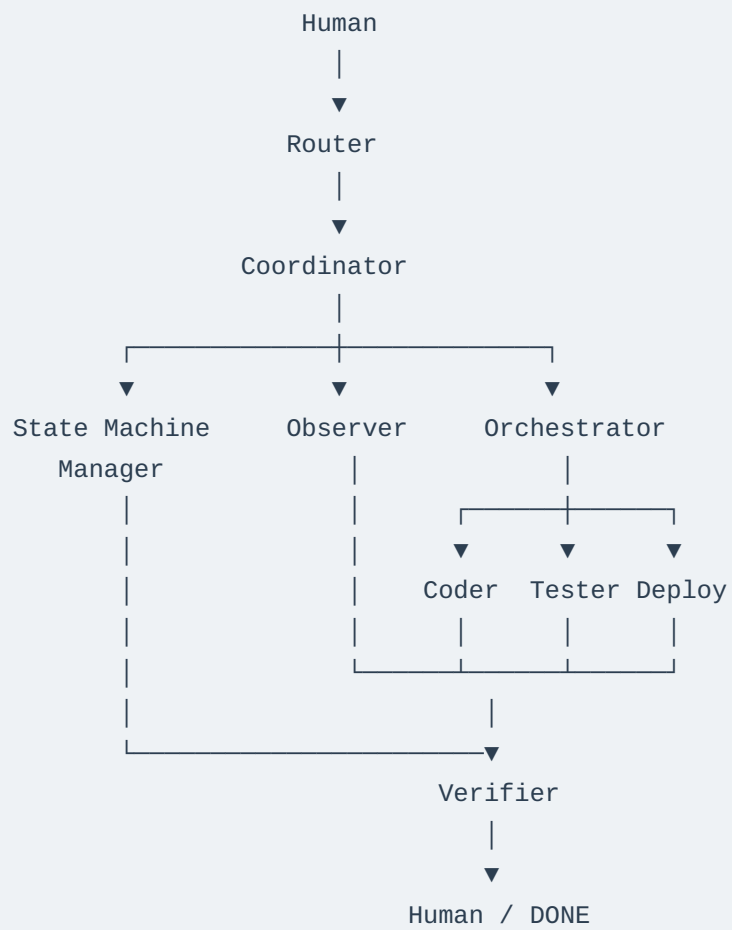
The Verifier determines whether the real result satisfies the task goal. It verifies observable outcomes, not merely the agent's claim of completion.

5.7 Human Gate

The Human Gate preserves human control over high-risk, irreversible, or subjective transitions, such as:

- production deployments;
 - database deletion;
 - permission changes;
 - large-scale refactoring;
 - visual acceptance;
 - business-rule confirmation.
-

6. Reference Architecture



7. State-Machine Model

7.1 Base Lifecycle

```
NEW
↓
ANALYZING
↓
READY_FOR_EXECUTION
↓
EXECUTING
↓
EXECUTION_DONE
↓
REVIEWING
↓
READY_TO_DEPLOY
↓
WAITING_HUMAN_APPROVAL
↓
DEPLOYING
↓
DEPLOYED
↓
VERIFYING
↓
VERIFIED
↓
DONE
```

7.2 Exceptional States

```
BLOCKED
FAILED
NEED_HUMAN_INPUT
DEPLOY_FAILED
VERIFICATION_FAILED
ROLLBACK_REQUIRED
REOPENED
```

For example:

```
VERIFYING
├── VERIFIED → DONE
└── FAILED
    ↓
    REOPENED
    ↓
    EXECUTING
```

7.3 Desired State and Observed State

ATCP recommends distinguishing:

```
Desired State
Observed State
```

For example:

```
desired_state: VERIFIED
observed_state: DEPLOYED
```

The task is not complete because deployment has not yet been verified.

If:

```
declared_state: DEPLOYED
observed_state: OLD_VERSION_RUNNING
```

the Observer should emit a state-inconsistency event.

8. Definition of Done

Different task types require different completion criteria.

8.1 Frontend Feature

- ☐ Code changed
- ☐ Lint / build passed
- ☐ Review completed
- ☐ Deployed
- ☐ Page verified
- ☐ No new critical console errors

8.2 Bug Fix

- ☐ Bug reproduced
- ☐ Fix implemented
- ☐ Original scenario verified
- ☐ Regression verification completed
- ☐ Deployed
- ☐ Production result confirmed

8.3 Database Change

- ☐ Change plan
- ☐ Risk assessment
- ☐ Backup
- ☐ Human approval
- ☐ Execution
- ☐ Data validation
- ☐ Rollback path confirmed

The state machine uses the applicable Definition of Done to determine whether transition to **DONE** is valid.

9. Traceability

Every task should have a unique Task ID, for example:

Task T-018

A complete chain might be:

```
User request
↓
T-018
↓
Coding Agent
↓
src/pages/device/index.tsx
↓
Git Diff
↓
Commit a82f31
↓
Build #218
↓
Deploy prod-20260903-01
↓
Verification #V-018
```

When a user inspects a page or artifact, the system should be able to answer:

- Which Task most recently changed it?
- Which agent performed the work?
- Why was it changed?
- Which files were changed?
- Was it deployed?
- Was it verified?

10. Human Attention Routing

ATCP should not require humans to continuously monitor every agent.

The system should route attention:

Active Tasks: 20

Normal: 15

Auto-recovering: 3

Need Human: 2

The user should see only:

Needs your attention:

1. T-031
Production deployment approval requested
2. T-044
UI visual verification required

The design objective is:

As the number of agents grows, the human's current working set should remain as small as possible.

11. Observer Detection Capabilities

The Observer may combine:

- language-model semantic analysis;
- state-machine rules;
- Git state;
- build results;
- test results;
- terminal output;
- deployed versions;
- logs;
- HTTP checks;

- time thresholds;
- retry counts.

For example:

```
same_action_failures >= 4
AND
observed_state unchanged
```

may trigger:

```
LOOP_SUSPECTED
```

Likewise:

```
Original Goal:
Deploy a frontend change

Current Actions:
Repeatedly modify SSH configuration
```

may trigger:

```
GOAL_DRIFT_SUSPECTED
```

12. Inter-Agent Communication

ATCP discourages transferring Agent A's complete conversation history to Agent B.

Instead, it recommends structured handoffs:

```
task_id: T-1024
goal: "Deploy the latest frontend build"
current_state: READY_TO_DEPLOY
```

```
known_facts:
  - build_passed
  - artifact_id: build-218
```

```
constraints:
  - use_existing_deploy_path
  - do_not_change_application_code
```

```
expected_output:
  - deployment_result
  - deployed_version
```

In short:

Share structured state, not an entire mind.

13. Minimum Viable Implementation

The first ATCP implementation does not need to become a complete agent platform.

A useful minimum is:

```
Task
+
State Machine
+
Trace
+
Observer
+
Human Approval
```

13.1 MVP Dashboard

Active Tasks: 12

Executing	4
Waiting Approval	2
Need Verification	3
Blocked	1
Failed	1
Done Today	8

Each task should display:

Task
Goal
Agent
Current State
Last Event
Changed Artifacts
Deploy Status
Verify Status
Need Human?

14. Non-Goals

ATCP v0.1 does not attempt to:

- define a universal language-model protocol;
- define a general AGI architecture;
- replace Git;
- replace CI/CD;
- replace existing project-management systems;
- delegate every decision to AI;
- require multiple models;
- require every logical role to be a separate agent.

Logical roles may be implemented through any combination of:

- software rules;
 - state machines;
 - one language model;
 - multiple language models;
 - human roles.
-

15. Design-Principle Summary

1. Task over Chat

Task is the unit of work; chat is an interface.

2. State over Memory

State belongs in the system, not in a person's head.

3. Trace over Recall

Results should be traceable instead of depending on memory.

4. Software Rules over LLM Guessing

Deterministic workflow belongs in state machines and software policy.

5. Isolation over Shared Context

Context should be isolated by default.

6. Observation over Blind Trust

Agent self-reports should be verified against observable state.

7. Human Governance over Full Autonomy

Humans should move from continuous operation to governance of critical decisions.

8. Attention Routing over Constant Monitoring

Only work requiring a human decision should consume human attention.

16. Future Directions

Future work may explore:

- an Agent Task Protocol;
 - an Agent Trace standard;
 - agent-observability metrics;
 - goal-drift detection;
 - agent context-contamination detection;
 - a Human Attention Scheduler;
 - agent conflict detection;
 - automatic Task DAG generation;
 - agent permission models;
 - secure agent sandboxes;
 - multi-project resource coordination;
 - agent SLAs;
 - agent task recovery;
 - distributed agent control planes;
 - integration with Git, CI, IDEs, and ticket systems.
-

17. Central Thesis

This whitepaper is not ultimately about:

How to run more agents at the same time.

It is about:

How humans can remain in control as the number of agents grows.

Agents can scale.

Human working memory cannot.

Therefore:

The next infrastructure layer for multi-agent systems is not only orchestration. It is a Task Control Plane.

18. Discussion

This is an early reference architecture, not a final standard.

Questions for discussion include:

1. Should Task State have a common standard?
 2. How much intervention authority should the Observer possess?
 3. Are Desired State and Observed State suitable abstractions for general-purpose agents?
 4. Should the Human Gate be modeled as a state, an event, or a policy?
 5. How should agent goal drift be defined?
 6. How can context contamination between agents be reduced?
 7. How should Human Cognitive Load be measured?
 8. How many agents can one person realistically govern?
 9. Should agents be allowed to modify Task State directly?
 10. Should the Task Control Plane be decoupled from the Agent Runtime?
-

License

Recommended licensing:

- This whitepaper: **CC BY 4.0**
- Future reference implementation: **Apache License 2.0** or **MIT License**

Citation

A formal release should add:

```
CITATION.cff
```

The project may also archive versioned releases with Zenodo to obtain a DOI.

Agent Task Control Plane — v0.1

Agents scale. Human working memory doesn't.